Module 3 — Request & Response Models using Pydantic (AI Engineer Perspective)

Learning Objectives

By the end of this module, you will be able to:

- Define **Pydantic models** to act as a rigorous "contract" for ML input data.
- Utilize **BaseModel** and **Python type hints** to perform automatic data validation and conversion.
- Implement **nested schemas** to handle complex, multi-feature AI input structures.
- Configure **Response Models** to ensure your model's output is clean, reliable, and production-ready.
- Contrast the "fragile" nature of notebook code with the "robust" nature of production-grade inference services.

Concept-by-Concept Explanations

1. What is Pydantic and Why AI Systems Require Strict Schemas

Pydantic is the "data part" of FastAPI. It is a library that uses standard Python type hints to define the **schema**—an abstract description of the shape and type of data your service receives and sends.

For an AI Engineer, a schema is a **rigorous mathematical contract**. Machine Learning models are not flexible with their inputs; a neural network expecting a 1×10 vector of floats will crash or produce nonsense if it receives 9 floats or a string.

- **Why this exists:** To ensure that data arriving from the network is converted from raw JSON into clean Python objects that your ML model can actually process.
- **Where it is used in AI systems:** It is used at the "entry point" of the inference service to guard the model from malformed data.
- **What breaks if this is missing:** Without schemas, you must write dozens of manual `if` statements to check every input. This leads to approximately 40% more human-induced bugs.

2. How Unvalidated Inputs Break ML Systems

In a Jupyter notebook, you often control the input directly. In a production API, anyone (or any other system) can send data to your model.

- **Shape Mismatch:** If your model expects four features (e.g., House Size, Age, Rooms, ZipCode) but only gets three, the matrix multiplication in the model will fail.
- **Missing Features:** If a mandatory feature for a **Fraud Detection** model (like `transaction_amount`) is missing, the model cannot compute a risk score.
- **Wrong Data Types:** If `house_age` is sent as the string `"ten"` instead of the integer `10`, the math logic in `scikit-learn` or `PyTorch` will raise a `TypeError`.

3. The Pydantic `BaseModel`

To define a schema, you create a class that inherits from Pydantic's `BaseModel`.

- **Why this exists:** It provides a template that FastAPI uses to validate the **Request Body**.
- **Where it is used in AI systems:** To define the "Feature Vector" or "Inference Payload".
- **What breaks if this is missing:** FastAPI won't know how to parse the incoming JSON, and you'll be forced to handle raw strings or dictionaries manually.

4. Field Typing (int, float, list, optional)

FastAPI uses standard Python types to decide how to validate data.

- **int/float:** Used for numerical features like `confidence_score` or `pixel_intensity`.
- **list:** Essential for sending arrays of features or batches of text to a model.
- **Optional:** Used when a feature is not strictly required for the model to run (e.g., an optional `user_comment` that doesn't affect the prediction).
- **Why this exists:** To perform **Automatic Conversion**. If a user sends a number as a string (e.g., `"5"`), Pydantic automatically converts it to an integer `5` before your model sees it.
- **Where it is used in AI systems:** Everywhere. Every model input must have a type (e.g., a `list[float]` for an embedding).
- **What breaks if this is missing:** Your model might receive a string when it expects a float, causing a crash deep inside a library like NumPy.

5. Nested Schemas

In AI, data is often hierarchical. For example, a **Image Metadata Inference** service might need a `CameraInfo` object inside a `MetadataRequest` object.

- **Why this exists:** To organize complex data into logical sub-groups.
- **Where it is used in AI systems:** Grouping related features, such as "User Profile Features" and "Transaction Features" for a **Fraud Detection** model.
- **What breaks if this is missing:** The data becomes a giant, flat list of variables that is difficult to maintain and document.

6. Response Models vs. Request Models

A **Request Model** is what the user sends *to* your model. A **Response Model** is what your API sends *back* to the user.

- **Why this exists:** To filter the output. You might want to hide internal model metrics (like `raw_logits` or `latency`) and only show the final `prediction`.
- **Where it is used in AI systems:** Formatting the prediction result into a standard, readable JSON format.
- **What breaks if this is missing:** You might accidentally leak sensitive internal data or provide a inconsistent output format that breaks the system consuming your API.

Code Examples with Line-by-Line Explanations

Example 1: Simple Feature Input (Price Prediction)

This model defines the features needed to predict a house price.

```
from pydantic import BaseModel #

class HouseFeatures(BaseModel): #
    square_feet: int #
    num_bedrooms: int #
    has_garden: bool = False #
```

Line-by-Line Explanation:

1. **`from pydantic import BaseModel`**: Imports the base class required to create schemas.
2. **`class HouseFeatures(BaseModel)`**: Defines our schema class. By inheriting from `BaseModel`, we tell FastAPI this is a data model.
3. **`square_feet: int`**: A required feature. If the user doesn't send this, FastAPI returns an error.
4. **`num_bedrooms: int`**: Another required feature.
5. **`has_garden: bool = False`**: This field is **Optional** because it has a **Default Value** (`False`). If the user doesn't send it, the model assumes the house has no garden.

ML Consumption: Your model code would receive an object `data` and access features via `data.square_feet`.

Example 2: ML-style Feature Vector (Fraud Detection)

This demonstrates how to send a list of numerical features (embeddings).

```
from typing import List #
from pydantic import BaseModel

class FraudInferenceRequest(BaseModel):
    transaction_id: str #
```

```
    feature_vector: List[float] #
```

Line-by-Line Explanation:

1. **`from typing import List`**: Imports the list type from Python's standard library.
2. **`transaction_id: str`**: An identifier for the transaction.
3. **`feature_vector: List[float]`**: This tells FastAPI to expect a list of numbers. FastAPI will validate that *every* item in that list is a float.

ML Consumption: This allows you to pass `feature_vector` directly into a model's `predict()` function as a list or NumPy array.

Example 3: Prediction Output Schema

This ensures the user gets a consistent response.

```
from pydantic import BaseModel

class PredictionResponse(BaseModel):
    label: str #
    probability: float #
```

Line-by-Line Explanation:

1. **`label: str`**: The final class name (e.g., "Fraud" or "Not Fraud").
2. **`probability: float`**: The confidence score of the model.

ML Consumption: Even if your model returns a messy dictionary with 20 different metrics, the **Response Model** ensures the user only sees these two cleaned fields.

Scenario-Based Explanations

Scenario: Fraud Detection Service

In a production system, a Fraud Detection API might receive a nested JSON. The **Nested Schema** allows you to group data:

- `UserGroup`: Includes `account_age` and `is_verified`.
- `TransactionGroup`: Includes `amount` and `location`. FastAPI validates all these layers automatically, ensuring your fraud model never processes an "empty" transaction.

Scenario: Price Prediction

Using **Field Typing**, you ensure that `square_feet` is always a number. If a user tries to send `"huge"` instead of `2500`, FastAPI automatically blocks the request with a "Value is not a valid integer" error before it ever reaches your expensive model logic.

Notebook vs. Production API: The Contrast

Feature	Notebook-style ML Code	Production API ML Code
Input Handling	Variables are defined in a cell; no validation.	Defined by Pydantic; strictly validated.
Error Management	Code crashes with a Python traceback if data is wrong.	API returns a clean, standard 422 error to the user.
Output Type	Prints objects or plots.	Returns a standardized JSON Response Model.
Reliability	Works for the author on their data.	Works for any user sending data that meets the "Contract".

Common Mistakes AI Engineers Make

- **Using Raw Dictionaries:** Not using a Pydantic model and trying to use `request: dict`. This loses all automatic validation and documentation benefits.
- **Missing Default Values:** Forgetting that if a field doesn't have a default value, it is **Required**. This can cause an API to break when users omit non-essential fields.
- **Ignoring Response Models:** Returning the raw output of a library like `PyTorch` or `TensorFlow`. These outputs often contain objects that cannot be converted to JSON, resulting in an "Internal Server Error".
- **Manual Type Conversion:** Writing code like `int(value)` inside your function instead of letting Pydantic's typing system handle it automatically.

Assignment: Build a House Price Predictor API

Task:

1. Create a schema named `PredictRequest` with four fields: `area` (float), `rooms` (int), `zip_code` (str), and an optional `year_built` (int, defaults to 2000).
2. Create a schema named `PredictResponse` with two fields: `estimated_price` (float) and `status` (str).
3. Implement a **POST** endpoint at `/predict` that:
    - Takes `PredictRequest` as the body.
    - Calculates a dummy price (e.g., `area * rooms * 100`).

    - Returns a `PredictResponse` object.
4. Run the server and use the `/docs` page to verify that sending a string for `area` triggers an automatic validation error.

Mini Project Idea: Production Inference API

Goal: Build a "Medical Diagnosis Inference API". Create a nested schema where the top level is `DiagnosisRequest`. Inside, include a nested `PatientProfile` (age, weight, gender) and a `VitalSigns` model (heart_rate, blood_pressure). Define a `DiagnosisResponse` that only returns a `risk_level` (str) and `recommendation` (str), hiding all the raw numerical VitalSigns from the final response.

Summary: Why this module is critical for AI Engineers

This module transitions you from being a "Model Researcher" to a "Systems Engineer." By mastering Pydantic and schemas, you ensure your model is **protected** from the chaos of real-world data. This is the difference between a model that only works on your computer and a **production-ready service** that can handle millions of requests with fewer bugs and clear documentation.